

Weather-Driven Delay Estimation

Check-in per titik (bukan tracking GPS terus-menerus), diikat ke rute spesifik yang dipilih user — menutup checklist proposal No. 13 (Route Duration and Delay Estimation).

STATUS: RENCANA — BELUM DIIMPLEMENTASI

Branch: fix/hackathon-readiness

Service: services/ai/route-planner

Tanggal: 22 Agustus 2026

Terkait: PROPOSAL_ALIGNMENT_CHECKLIST — item No. 13

Riwayat revisi rencana ini (dalam sesi yang sama):

- v1 — tabel multiplier sintetis murni (ditolak, dianggap kurang)
- v2 — 1 label cuaca per keseluruhan trip dari mobile app (kurang presisi)
- v3 — GPS trace kontinu ala Google Maps (butuh app selalu aktif di background)
- v4 (final) — **check-in per titik**, diikat ke rute spesifik yang dipilih user

1. Latar Belakang & Evolusi Rencana

Checklist No. 13 menemukan `expected_delay_hours` selalu 0 di setiap prediksi rute normal — tidak pernah menyesuaikan berdasarkan cuaca yang terdeteksi.

ARAHAN PENGGUNA, DISEMPURNAKAN BEBERAPA PUTARAN

Dimulai dari penolakan terhadap tabel sintetis murni → diarahkan ke data nyata dari "mobile" → diperjelas jadi analogi Google Maps (tahu durasi & kondisi hujan dari kendaraan yang benar-benar jalan) → disederhanakan jadi **check-in per titik** (bukan tracking GPS terus-menerus, karena itu butuh aplikasi yang selalu aktif) → disempurnakan lagi: **tracking harus diikat ke rute spesifik yang user pilih** dari hasil `/predict-route`, bukan sekadar "pengiriman ini" secara abstrak.

DI LUAR CAKUPAN RENCANA INI

Membangun kanal pelaporan spesifik (aplikasi, gateway SMS, bot WhatsApp, form web, integrasi gerbang tol) — rencana ini hanya membangun **endpoint backend** yang bisa dipanggil kanal mana pun begitu dibangun.

2. Prinsip Desain

2.1 Rute darat belum pernah dapat cuaca nyata sama sekali

`enrichment_service._resolve_conditions()` untuk moda `darat` selalu mengembalikan dict tetap (`weather_condition: "Berawan"`) — BMKG tidak punya produk cuaca darat. Open-Meteo sudah dipakai untuk suhu ambient kargo pasif — parameter `hourly`-nya menerima banyak variabel sekaligus, jadi menambah `precipitation, weathercode` ke request yang sama adalah perluasan berisiko rendah.

2.2 Taksonomi tingkat keparahan cuaca yang kecil, pakai ulang kosakata yang sudah ada

Kode WMO dari Open-Meteo dipetakan ke kosakata `weather_condition` yang sama persis dengan BMKG & data training (`Cerah ... Hujan Badai`). Lapis kedua mengelompokkannya jadi 4 tingkat keparahan delay (`normal`, `hujan_ringan`, `hujan_lebat`, `badai`) — sengaja kasar supaya data nyata cepat "cukup untuk dipercaya".

2.3 Data mentah: check-in per titik, bukan tracking GPS terus-menerus

Setelah pertimbangan biaya-vs-manfaat, tracking GPS kontinu (butuh app selalu aktif di background, izin lokasi, konsumsi baterai) disederhanakan jadi **beberapa check-in diskrit** per perjalanan — minimal keberangkatan & tiba, idealnya ditambah 1–2 titik tengah (rest area, gerbang tol). Sopir tap tombol, atau kanal apa pun (SMS, bot WhatsApp, scan gerbang tol) melaporkan 3 angka yang sama: lat, lon, waktu.

Lapis	Isi	Analogi fitur sebelumnya
Mentah	Tabel baru <code>shipment_checkpoints</code> — (<code>shipment_id</code> , <code>lat</code> , <code>lon</code> , <code>recorded_at</code> , <code>checkpoint_label</code>), endpoint <code>POST /shipments/{id}/checkpoints</code> .	Analog <code>actual_delay_hours</code> di <code>shipment_records</code> — audit mentah per kejadian.
Analisis segmen	Dipicu saat <code>PATCH /shipments/{id}/outcome</code> — jalan tiap pasangan checkpoint berurutan, hitung kecepatan aktual vs nominal, cari cuaca di titik tengah/waktu tengah segmen.	Konsep baru, tapi memakai fungsi yang sudah ada (<code>haversine_km</code> , <code>Open-Meteo</code>).
Agregat	Tabel baru <code>weather_delay_stats</code> , kunci (<code>transport_mode</code> , <code>weather_severity</code>).	Sama persis bentuknya dengan <code>corridor_baseline_stats</code> .

KETERBATASAN YANG DIAKUI TERBUKA

Satu segmen antar-checkpoint bisa sepanjang puluhan km dan melewati lebih dari satu kondisi cuaca sungguhan — analisis ini mengasumsikan cuaca di titik tengah/waktu tengah mewakili seluruh segmen. Lebih banyak checkpoint per perjalanan (mis. tiap rest area) langsung memperbaiki ini — ini trade-off biaya pengumpulan data, bukan keterbatasan kode.

*Analisis segmen untuk sekarang hanya berjalan untuk moda **darat** — moda laut/kombinasi butuh klasifikasi checkpoint mana yang di darat vs di laut (perlu pencocokan terhadap batas-batas leg route), dicatat sebagai tindak lanjut, tidak menghalangi rencana ini.*

2.4 BARU: Tracking diikat ke rute spesifik yang user pilih

DITAMBAHKAN SESUAI MASUKAN TERBARU

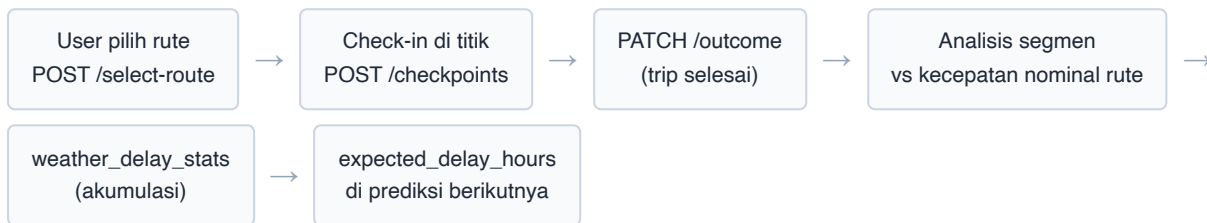
`/predict-route` mengembalikan satu rute rekomendasi *dan* beberapa alternatif — user/dispatcher yang pilih salah satu untuk benar-benar dijalankan, dan pilihan itu belum tentu yang direkomendasikan. Checkpoint harus diikat ke rute spesifik itu, bukan sekadar "pengiriman ini" secara umum — supaya (1) baseline yang dibandingkan benar rute yang dilewati, dan (2) kecepatan nominal pembanding memakai estimasi ORS untuk rute itu sendiri (`distance_km / estimated_duration_hours`), bukan satu angka pukul rata untuk semua rute darat — jalan tol dan jalan kampung jelas beda kecepatan wajarnya, dan ORS sudah tahu bedanya.

- `shipment_records` dapat kolom baru `selected_route_id` — diisi lewat endpoint baru `POST /shipments/{id}/select-route`, tervalidasi harus cocok dengan `recommended_route_id` atau salah satu ID di `alternative_routes` milik shipment tersebut (kalau tidak cocok → 422).
- Analisis segmen mengambil rute yang dipilih itu dari `prediction_snapshot`, hitung `nominal_speed_kmh = distance_km / estimated_duration_hours` khusus rute itu. Kalau belum pernah pilih rute (checkpoint dikirim tanpa lewat `/select-route` dulu), jatuh kembali ke konstanta pukul-rata `45 km/h` — jadi tetap opsional, tidak memblokir pemakaian fitur checkpoint sama sekali.

2.5 Serving time: blend bootstrap + data nyata, masuk ke `expected_delay_hours`

Tingkat keparahan	Bootstrap default (jam)	≈ menit
● normal	0.00	0 menit
● hujan_ringan	0.17	~10 menit
● hujan_lebat	0.42	~25 menit
● badai	1.00	~60 menit

Placeholder ilustratif, ditandai eksplisit di API sebagai belum tervalidasi — digeser otomatis oleh rata-rata nyata begitu jumlah sampel mencukupi (pola Corridor Baseline Learning).



2.6 Train/serve skew — didokumentasikan, bukan diselesaikan sekarang

KONSISTEN DENGAN KEPUTUSAN SEBELUMNYA

Data training darat tidak pernah berkorelasi cuaca (wave_category selalu "Tenang" untuk darat), sementara laut/kombinasi sudah. Mengaktifkan delay cuaca nyata untuk darat berarti model menerima pola input yang belum pernah dilihat khusus untuk darat — didokumentasikan eksplisit, retrain dicatat sebagai tindak lanjut terpisah, bukan bagian rencana ini.

3. Rincian Implementasi per Komponen

3.1 Pengambilan cuaca darat — temperature_service.py

```

def _fetch_open_meteo_hourly(client, lat, lon, target_time) -> dict | None:
    # params={"hourly": "temperature_2m,precipitation,weathercode", ...}
    # return {"temp_c":..., "precipitation_mm":..., "weathercode":...}
# fetch_ambient_temp_c() jadi wrapper tipis (perilaku lama tetap sama)
# fetch_weathercode() wrapper baru
  
```

3.2 Pemetaan kode WMO — weather_service.py

```

OPEN_METEO_WEATHERCODE_TO_CONDITION = {
    0: "Cerah", 1: "Cerah Berawan", 2: "Cerah Berawan", 3: "Berawan",
    45: "Berawan Tebal", 48: "Berawan Tebal",
    51: "Hujan Ringan", 53: "Hujan Ringan", 55: "Hujan Ringan", 56: "Hujan Ringan", 57: "Hujan Ringan",
    61: "Hujan Sedang", 63: "Hujan Sedang",
    65: "Hujan Lebat", 66: "Hujan Lebat", 67: "Hujan Lebat", 80: "Hujan Lebat", 81: "Hujan Lebat",
    82: "Hujan Badai", 95: "Hujan Badai", 96: "Hujan Badai", 99: "Hujan Badai",
}
def weathercode_to_condition(code): return OPEN_METEO_WEATHERCODE_TO_CONDITION.get(code, "Berawan")

```

3.3 Resolusi kondisi darat — `enrichment_service.py`

Cabang darat di `_resolve_conditions()` : sampling titik sepanjang `candidate["geometry"]` , panggil `fetch_weathercode()` , petakan ke `weather_condition` . Gagal apa pun → jatuh ke `LAND_DEFAULT_CONDITIONS` yang sudah ada, tanpa mode kegagalan baru.

3.4 BARU: Pemilihan rute & check-in — `db.py` , `shipment_schema.py` , `routers/shipments.py`

```

@router.post("/shipments/{shipment_id}/select-route", response_model=ShipmentRecord)
async def select_route(shipment_id, body: RouteSelection) -> dict:
    # 422 kalau route_id tidak cocok dengan recommended_route_id
    # atau salah satu alternative_routes[].route_id di prediction_snapshot

shipment_checkpoints = Table(
    "shipment_checkpoints", metadata,
    Column("id", Integer, primary_key=True, autoincrement=True),
    Column("shipment_id", String, nullable=False, index=True),
    Column("lat", Float, nullable=False),
    Column("lon", Float, nullable=False),
    Column("recorded_at", DateTime(timezone=True), nullable=False),
    Column("checkpoint_label", String, nullable=True),
)

@router.post("/shipments/{shipment_id}/checkpoints", status_code=202)
async def submit_checkpoint(shipment_id, body: CheckpointBatch) -> dict:
    return {"checkpoints_received": len(body.points)}

```

Endpoint checkpoint biasanya dipanggil 1x per check-in (tap tombol, scan gerbang tol); bentuk batch cuma mengakomodasi kanal yang menyimpan beberapa check-in dulu lalu sync sekaligus (mis. tidak ada sinyal di rest area).

3.5 Analisis segmen — `historical_baseline.py`

```

NOMINAL_SPEED_KMH_DARAT = 45.0 # fallback saja, mirror training/synthetic_corridors.py

def _nominal_speed_kmh(shipment: dict) -> float:
    """Utamakan kecepatan estimasi ORS dari rute yang dipilih sendiri
    (distance/duration dari prediction_snapshot) -- baru fallback ke konstanta
    kalau belum ada rute yang dipilih."""
    route_id = shipment.get("selected_route_id")
    if not route_id:
        return NOMINAL_SPEED_KMH_DARAT
    snapshot = shipment["prediction_snapshot"]
    candidates = [snapshot["recommended_route"], *snapshot["alternative_routes"]]
    route = next((r for r in candidates if r["route_id"] == route_id), None)
    if route is None or route["estimated_duration_hours"] <= 0:
        return NOMINAL_SPEED_KMH_DARAT
    return route["distance_km"] / route["estimated_duration_hours"]

async def process_trip_weather_delay(client, shipment: dict) -> None:
    """Best-effort; dipanggil dari report_outcome(). Jalan tiap pasangan
    checkpoint berurutan, hitung delay vs kecepatan nominal rute, cari cuaca
    di titik tengah/waktu tengah, masukkan ke weather_delay_stats."""
    if shipment["transport_mode"] != "darat":
        return
    nominal_speed_kmh = _nominal_speed_kmh(shipment)
    points = db.select_checkpoints(shipment["shipment_id"])
    for p1, p2 in zip(points, points[1:]):
        distance_km = haversine_km(p1["lat"], p1["lon"], p2["lat"], p2["lon"])
        elapsed_hours = (p2["recorded_at"] - p1["recorded_at"]).total_seconds() / 3600
        expected_hours = distance_km / nominal_speed_kmh
        delay_hours = max(0.0, elapsed_hours - expected_hours)
        if delay_hours <= 0:
            continue
        # cari cuaca di titik tengah/waktu tengah, lalu upsert ke weather_delay_stats
        ...

```

3.6 Agregat & blend serving-time — db.py, historical_baseline.py

```

weather_delay_stats = Table(
    "weather_delay_stats", metadata,
    Column("transport_mode", String, primary_key=True),
    Column("weather_severity", String, primary_key=True),
    Column("sample_count", Integer, nullable=False, default=0),
    Column("delay_hours_sum", Float, nullable=False, default=0.0),
    Column("updated_at", DateTime(timezone=True), nullable=False),
)

MIN_SAMPLES_FOR_WEATHER_BLEND = 3
FULL_TRUST_WEATHER_SAMPLES = 20

def estimate_weather_delay_hours(transport_mode, weather_condition) -> tuple[float, str]:
    """Return (jam, 'bootstrap' | 'learned')."""
    ...

```

3.7 Sambungan ke enrich_candidate()

```

weather_delay_hours, weather_delay_data_quality = estimate_weather_delay_hours(
    candidate["transport_mode"], conditions["weather_condition"]
)
total_delay_hours = expected_delay_hours + weather_delay_hours
# response: "expected_delay_hours": total_delay_hours (field sama, kini sadar cuaca)
#           "weather_delay_hours": weather_delay_hours (baru, transparan)
#           "weather_delay_data_quality": weather_delay_data_quality (baru)

```

4. Perubahan Skema API

Schema	Perubahan
RouteCandidate (route_schema.py)	+ weather_delay_hours: float + weather_delay_data_quality: Literal["bootstrap","learned"]
ShipmentRecord (shipment_schema.py)	+ selected_route_id: str None
(baru) RouteSelection	{"route_id": str} — body untuk POST /shipments/{id}/select-route
(baru) CheckpointReport / CheckpointBatch	{"lat","lon","recorded_at","checkpoint_label"?} — body untuk POST /shipments/{id}/checkpoints

5. Rencana Pengujian

- `weathercode_to_condition()` — tiap bucket WMO + fallback kode tak dikenal.
- `_weather_severity()`, `estimate_weather_delay_hours()` (bootstrap vs learned), `_nominal_speed_kmh()` (rute dipilih vs fallback), dan matematika delay per segmen — semua unit test murni.
- Extend `test_predict_route_e2e.py`: `weather_delay_hours ≥ 0`, `weather_delay_data_quality` valid.
- Baru `test_shipment_checkpoints_e2e.py` (butuh Postgres): prediksi rute → `/select-route` → beberapa `/checkpoints` (kecepatan jauh di bawah nominal rute) → `PATCH /outcome` → pastikan `weather_delay_stats` bertambah → prediksi ulang, pastikan `weather_delay_data_quality` bergerak ke "learned". Juga tes `/select-route` menolak `route_id` yang tidak ada di `prediction_snapshot` (422).
- Tes fallback tanpa DB: paksa `db.get_weather_delay_stats` error, pastikan tetap 200 dengan "bootstrap".

6. Daftar File yang Akan Disentuh

[diubah] `services/ai/route-planner/app/services/temperature_service.py`

[diubah] `services/ai/route-planner/app/services/weather_service.py`

[diubah] `services/ai/route-planner/app/services/enrichment_service.py`

[diubah] `services/ai/route-planner/app/services/historical_baseline.py`

[diubah] `services/ai/route-planner/app/core/db.py`

[diubah] `services/ai/route-planner/app/schemas/route_schema.py`

[diubah] `services/ai/route-planner/app/schemas/shipment_schema.py`

[diubah] `services/ai/route-planner/app/routers/shipments.py`

[diubah] `services/ai/route-planner/app/services/shipment_service.py`

[diubah] `services/ai/route-planner/tests/test_predict_route_e2e.py`

[baru] `services/ai/route-planner/tests/test_shipment_checkpoints_e2e.py`

7. Verifikasi Setelah Implementasi

1. `pytest tests/` tanpa Postgres — tes no-DB lulus; lalu `pytest tests/golden_demo/` .
2. `docker compose up -d postgres` , jalankan tes checkpoint/pembelajaran cuaca.
3. Manual: `POST /predict-route` darat di lokasi/waktu hujan — `weather_delay_hours > 0` , kualitas `"bootstrap"` . Buat shipment, `POST /select-route` , kirim beberapa checkpoint lambat, `PATCH /outcome` , ulangi beberapa trip, pastikan kualitas berubah jadi `"learned"` dan nilainya bergeser ke rata-rata nyata.